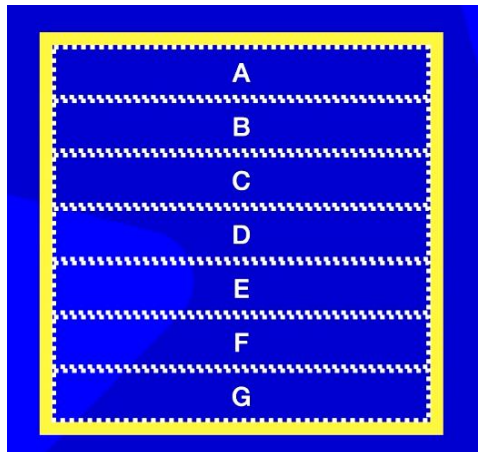


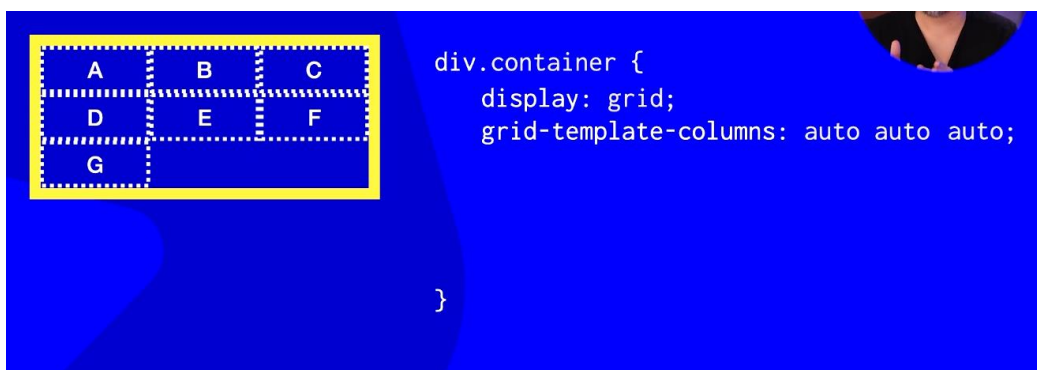
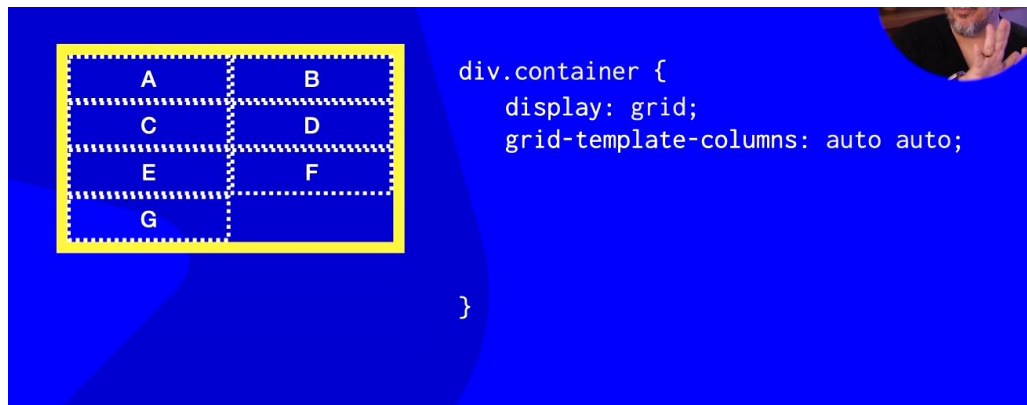
Grid

Propriedades do Container do Grid Layout

No exemplo abaixo, a parte amarela é a nossa div da classe contêiner e os pontilhados são div da classe item.

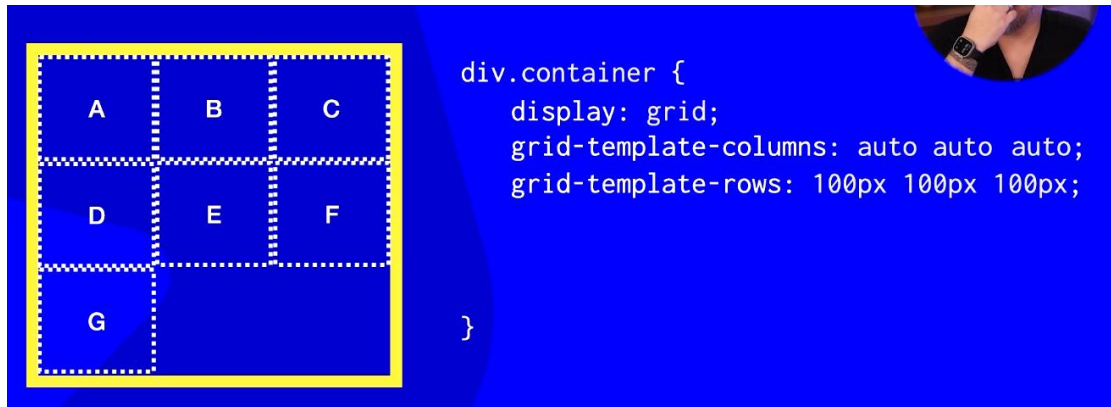


Quando você só muda o div contêiner para “display: grid”, o resultado visual será o mesmo, pois ele, por padrão, considera somente uma coluna. Entretanto, quando você usa o atributo “grid-template-columns” e o transforma em “auto auto”, ele cria duas colunas, como no exemplo a seguir:

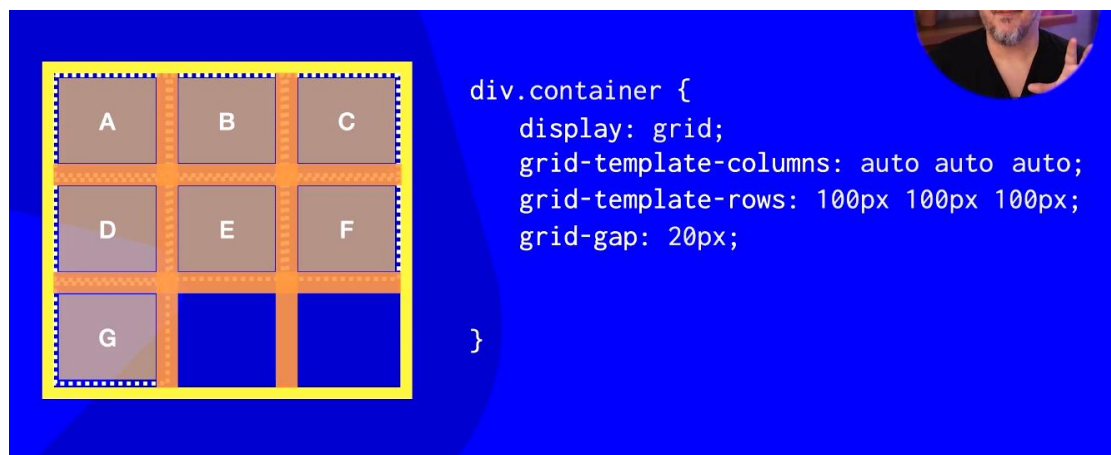


Se você colocar “auto auto auto”, como no exemplo acima, ele criará 3 colunas. Ademais, cada um desses itens tem a mesma largura e altura.

O grid-template-rows modifica a altura das linhas.



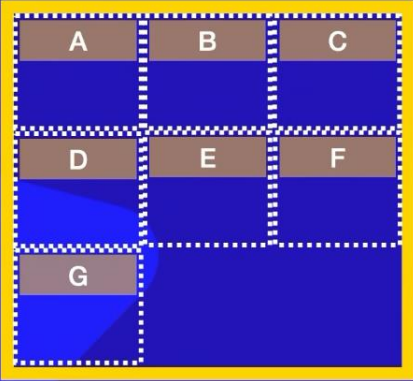
Grid-gap



Align em Grid, quer dizer alinhamento vertical, seja align-items ou align-content.

Justify, quer dizer alinhamento horizontal, seja justify-content ou justify-items.

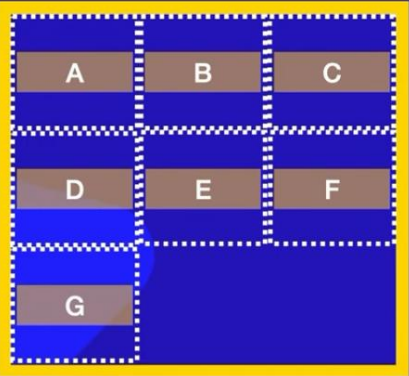
O padrão para o **align-items** é o valor “normal” que é o mesmo que stretch. O “stretch” estica verticalmente os itens até as grid lines. Também temos o valor “start”, que vai colar os itens em cima. Exemplo:



A 3x3 grid layout with a yellow border. The first column contains three items labeled A, D, and G. The second column contains two items labeled B and E. The third column contains one item labeled F. The grid is set to `align-items: start`, meaning all items are aligned to the top of their respective rows. Item G is at the bottom of the first column.

```
div.container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  grid-template-rows: 100px 100px 100px;  
  align-items: start;  
}
```

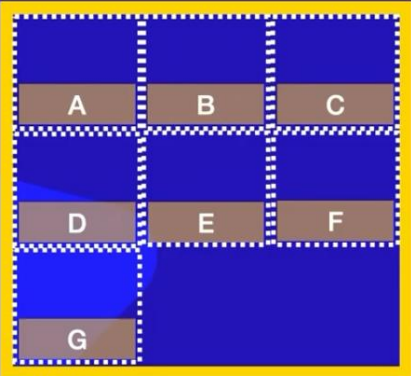
O valor “center”, vai deixar o item verticalmente no centro:



A 3x3 grid layout with a yellow border. The first column contains three items labeled A, D, and G. The second column contains two items labeled B and E. The third column contains one item labeled F. The grid is set to `align-items: center`, meaning all items are vertically centered within their rows. Item G is at the bottom of the first column.

```
div.container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  grid-template-rows: 100px 100px 100px;  
  align-items: center;  
}
```

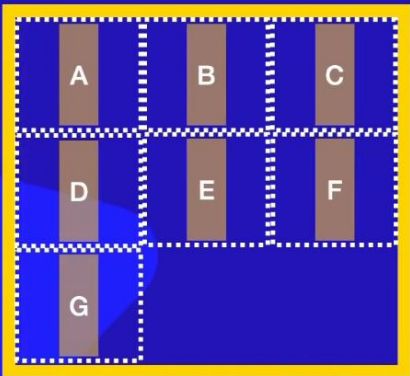
O valor “end”, vai deixar o item verticalmente no final:



A 3x3 grid layout with a yellow border. The first column contains three items labeled A, D, and G. The second column contains two items labeled B and E. The third column contains one item labeled F. The grid is set to `align-items: end`, meaning all items are aligned to the bottom of their respective rows. Item G is at the bottom of the first column.

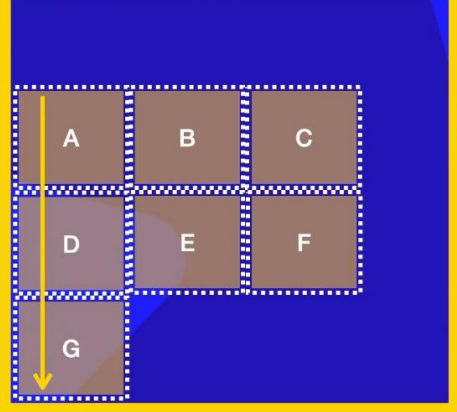
```
div.container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  grid-template-rows: 100px 100px 100px;  
  align-items: end;  
}
```

O **justify-items** têm os mesmos valores (stretch, start, center e end), mas mudará horizontalmente. Exemplo:



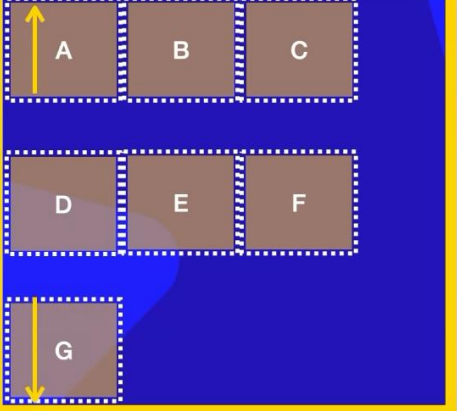
```
div.container {
  display: grid;
  grid-template-columns: auto auto auto;
  grid-template-rows: 100px 100px 100px;
  align-items: stretch;
  justify-items: center;
}
```

O **align-content** cuida do alinhamento dos itens como um conjunto. Ele tem os valores `start`, que deixa os itens colados lá em cima; `center`, que centraliza o conteúdo; o `end`, que deixará ele colado embaixo; e o `stretch`, que colará tanto na linha de baixo quanto de cima. Exemplo:



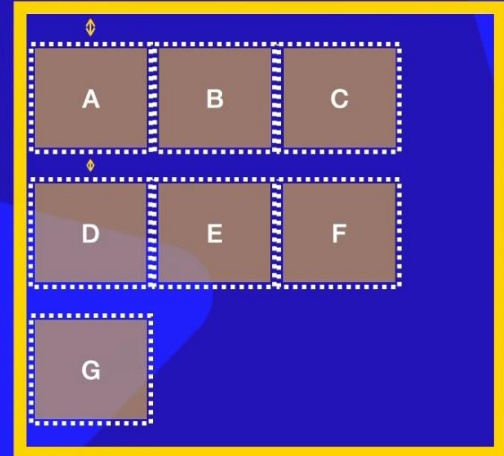
```
div.container {
  display: grid;
  grid-template-columns: auto auto auto;
  grid-template-rows: 100px 100px 100px;
  align-items: stretch;
  justify-items: stretch;
  align-content: end;
}
```

Também há o valor `space-between`:



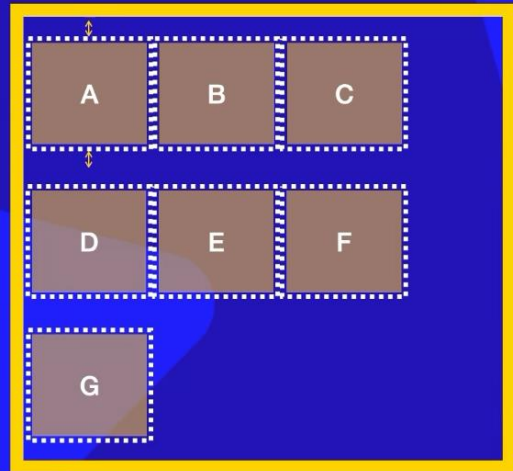
```
div.container {
  display: grid;
  grid-template-columns: auto auto auto;
  grid-template-rows: 100px 100px 100px;
  align-items: stretch;
  justify-items: stretch;
  align-content: space-between;
}
```

Space-evenly:



```
div.container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  grid-template-rows: 100px 100px 100px;  
  align-items: stretch;  
  justify-items: stretch;  
  align-content: space-evenly;  
}
```

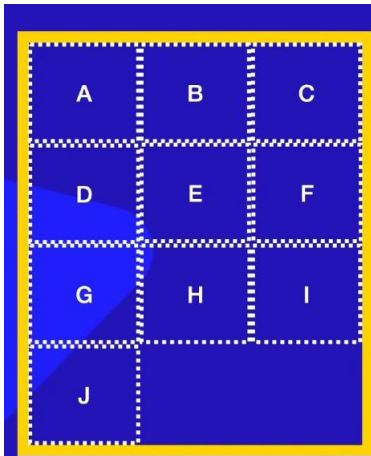
O “space-around”, diferente do “space-evenly”, deixa um espaço maior entre os itens do centro. Exemplo:



```
div.container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  grid-template-rows: 100px 100px 100px;  
  align-items: stretch;  
  justify-items: stretch;  
  align-content: space-around;  
}
```

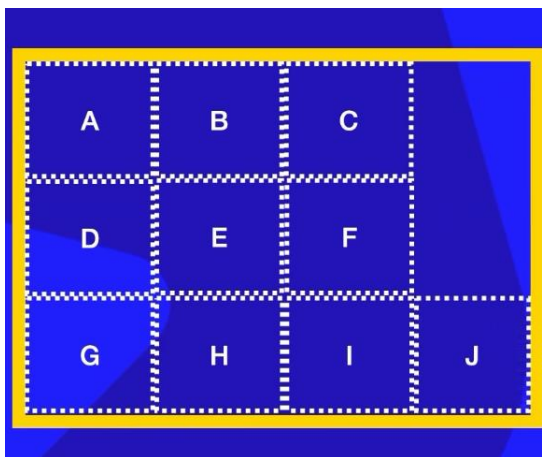
O **justify-content** tem valores iguais ao align-content, mas os muda horizontalmente. São eles: stretch, start, end, center, space-around, space-between e space-evenly.

O **grid-auto-row** faz com que, caso seja adicionado uma outras linhas no código pelo back-end, ela surja com uma dimensão preestabelecida. Exemplo:



```
div.container {
  display: grid;
  grid-template-columns: auto auto auto;
  grid-template-rows: 100px 100px 100px;
  grid-auto-row: 100px;
}
```

O **grid-auto-column** faz a mesma coisa, porém para colunas. Exemplo:



```
div.container {
  display: grid;
  grid-template-columns: auto auto au
  grid-template-rows: 100px 100px 100
  grid-auto-row: 100px;
  grid-auto-column: 100px;
}
```

As propriedades align-items e justify-items podem ser resumidas pelo “place-items”, assim como as propriedades align-content e justify-content podem ser resumidas pelo “place-content”. Todas na ordem citada, exemplo:

```
align-items: stretch; > place-items
justify-items: center; >
align-content: start; > place-content
justify-content: end; >
```

```
place-items: stretch center;
place-content: start end;
```

Também podemos resumir o `grid-template-rows` e o `grid-template-columns`, nessa mesma ordem, por **grid-template**. Exemplo:

```
display: grid;  
grid-template-columns: auto auto auto;  
grid-template-rows: 100px 100px 100px;
```

```
grid-template: 100px 100px 100px /  
               auto auto auto;
```

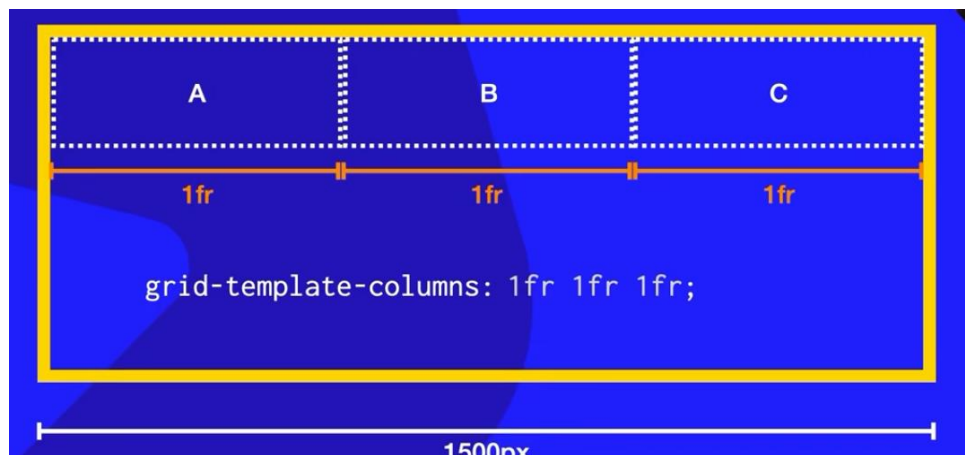
O nosso “`grid-template-columns: auto auto auto`” pode ser trocado por “**grid-template-columns: repeat(3, auto)**”. Exemplo:

```
div.container {  
  display: grid;  
  grid-template-columns: repeat(3, auto);  
  grid-template-rows: repeat(3, 100px);  
}
```

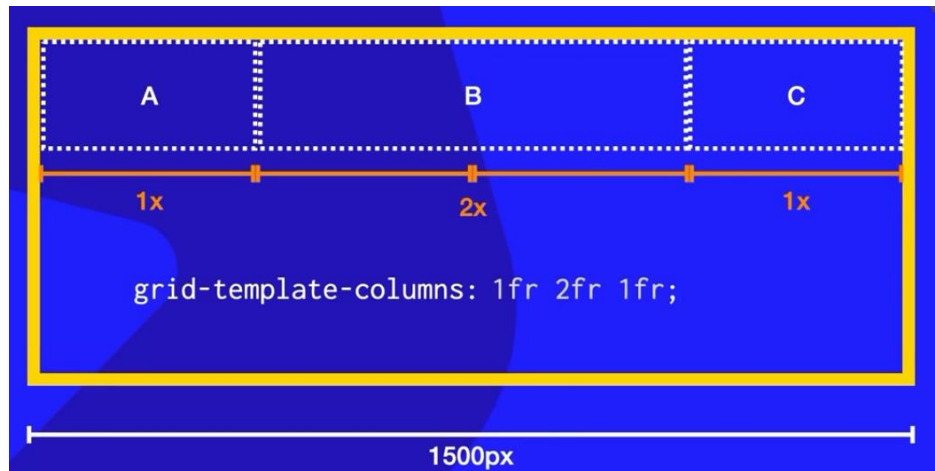
Podemos resumir ainda mais usando o `grid-template`:

```
grid-template:  
  repeat(3, 100px) / repeat(3, auto);
```

Unidade Fracional: quando for usar Grid, use separando os itens por frações ao invés do “auto”. Exemplo:



No exemplo acima, para deixar o item “B” com o dobro dos itens “A” e “C”, utilizando frações, você pode colocar:



No exemplo acima, nós temos 4 frações ($1\text{fr} + 2\text{fr} + 1\text{fr} = 4\text{fr}$), cada uma com 375px, com a célula “B” tendo 750px.

No exemplo abaixo, a porcentagem (%) é calculada de acordo com o contêiner total, mas a fração é com que sobrou do contêiner depois das modificações. Exemplo:

